Homework2 CS5800

Rayan Hassan

January 2025

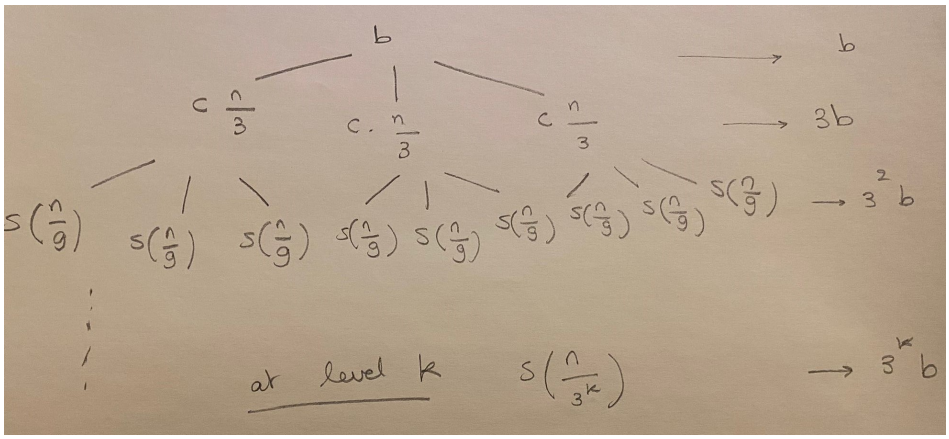
1 Exercise 1

1.1

Using a recursion tree, we get the following:

At level 0 (root): cost is b
At level 1: cost is 3b and size of each subproblem is $\frac{n}{3}$
At level 2: cost is 3^2b and size of each subproblem is $\frac{n}{9}$
At level k: cost is 3^kb and size of each subproblem is $\frac{n}{3^k}$ which means that $k = \log_3 n$

Please see the following figure for more explanation:



This means that the total cost is equal to:
$S(n) = b + 3b + 3^2b + ... + 3^{\log_3 n}b = b(1 + 3 + 3^2 + ... + 3^{\log_3 n})$ (geometric series)
Therefore $S(n) = b\frac{3^{\log_3 n+1}-1}{3-1}$
But $3^{\log_3 n+1} = 3 * 3^{log_3 n} = 3n$
So S(n) becomes: $S(n) = b\frac{3n-1}{2} = \frac{3b}{2}n - \frac{1}{2}b$.

This means that for $S(n) \leq Cn$ where $C > 0$. So $S(n) = O(n)$.
Similarly, $S(n) \geq Kn$ where $K > 0$. So $S(n) = \Omega(n)$.
Therefore, $S(n) = \Theta(g(n))$ where $g(n) = n$

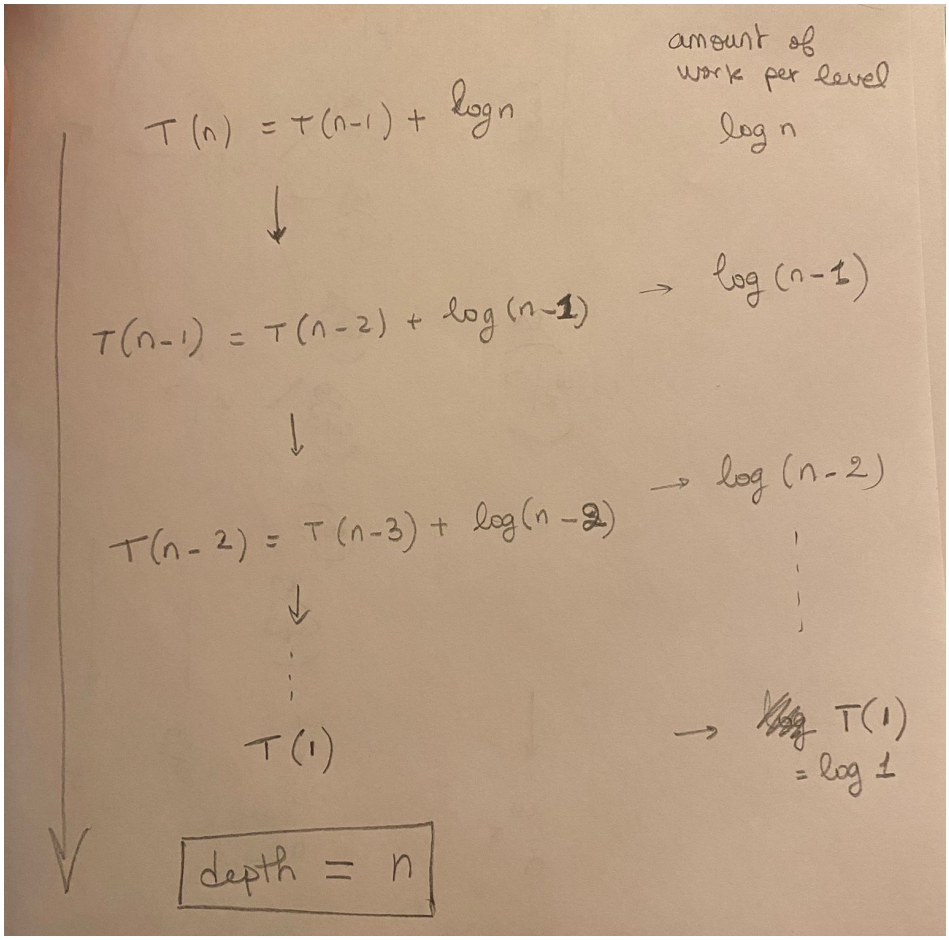
1.2

We want to prove that $S(n) \geq cg(n)$ where $c > 0$ and $n \geq n_1$.
Inductive hypothesis: $S(k) \geq ck$ for all $k < n$.
Base case: For n=1, $S(1) = 3S(\frac{1}{3}) + b \simeq b$. Which means if c is small enough, $S(1) \geq c.1$ (holds).
Substitute: $S(n) \geq 3(c.\frac{n}{3}) + b = cn + b$.
Therefore $S(n) \geq cn$ which means $S(n) = \Omega(n)$

2 Exercise 2

2.1

The following picture shows the corresponding recursion tree.



As seen in the image, the depth of the tree is equal to n and the work at each level is $\log n$, $\log(n-1)$, $\log(n-2)$, ..., $log1$ (base case).
Therefore, we get that $T(n) = \log n + \log(n-1) + \log(n-2) + ... + T(1)$.
By exaggerating we get: $T(n) \leq \log n + \log n + ... + \log n = n \log n$ (upper bound). So $T(n) = O(n \log n)$

Now by underestimating, we get: $T(n) \geq \log \frac{n}{2} + \log \frac{n-1}{2} + ...$
So $\sum_{k=1}^{n} \log k \geq \frac{n}{2} \log(\frac{n}{2})$.
But $log \frac{n}{2} = \log n - \log 2$, which means that:
$T(n) \geq \frac{n}{2}(\log n - \log 2) = \frac{n}{2} \log n - \frac{n}{2} \log 2$. For large n, the second term becomes

negligible. So $T(n) \geq \frac{n}{2} \log n$.
Therefore, $T(n) \geq Kn \log n (K > 0)$, so $T(n) = \Omega(n \log n)$. Finally we can say that $T(n) = \Theta(n \log n)$.

2.2

Hypothesis: $T(k) \leq ck \log k$.
Base case: $T(2) = T(1) + \log 2 \leq c \log 1 + \log 2 = \log 2$. And $\log 2 \leq c \log 2$ (holds).
Induction: (Substitute)
$T(n) \leq T(k-1) + \log k = c(k-1) \log(k-1) + \log k = (ck - c) \log(k-1) + \log k = ck \log(k-1) - c \log(k-1) + \log k$. By exaggerating we get: $T(n) \leq ck \log(k-1) - ck \log(k-1) + k \log k$. Therefore, $T(n) \leq k \log k$. So $T(n) = O(n \log n)$

3 Exercise 3

With $n = 2^m$, we get that $T(2^m) = T(\sqrt{2^m}) + 1 = T(2^{\frac{m}{2}}) + 1$.
Now let $S(m) = T(2^m)$. We get $S(m) = S(\frac{m}{2}) + 1$.
We can use recursion tree to solve this:
At the root, $S(m) = S(\frac{m}{2}) + 1$
At level 1, $S(\frac{m}{2}) = S(\frac{m}{4}) + 1$
At level 2, $S(\frac{m}{4}) = S(\frac{m}{8}) + 1$
(note that the work at each of these levels is 1). And this continues until we reach the base case where $\frac{m}{2^k} = 1$, which means $k = \log_2(m)$. The depth of the tree is therefore $\log_2(m)$. The total work is $1 + 1 + 1 + ... = \log_2(m)$. Therefore $S(m) = \log_2(m)$. Given that $n = 2^m$, $m = \log_2(n)$.
This means that $S(m) = T(2^m) = T(n) = \log_2(\log_2(n))$.
Therefore: $T(n) = \Theta(\log_2(\log_2(n)))$.

4 Exercise 4

4.1

$T(n) = 10T(\frac{n}{3})+\Theta(n^2 \log^5 n)$. In this case $a = 10, b = 3, f(n) = \Theta(n^2 \log^5 n), \log_b(a) = \log_3(10) \approx 2.1 > 2$. Therefore $T(n)$ is dominated by the recursion term. So $T(n) = \Theta(n^{\log_3(10)}) = \Theta(n^{2.1})$.

4.2

$T(n) = 256T(\frac{n}{4})+\Theta(n^4 \log^4 n)$. In this case $a = 256, b = 4, f(n) = \Theta(n^4 \log^4 n), \log_b(a) = \log_4(256) = 42$. Therefore the polynomial term n^4 matches $\log_4(256)$ but f(n) grows slightly larger since it includes an extra $\log^4 n$. So $T(n) = \Theta(n^4 \log^4 n)$.

4.3

$T(n) = T(\frac{19n}{72}) + \Theta(n^2)$. In this case $a = 1, b = \frac{72}{19}, f(n) = \Theta(n^2), \log_b(a) = \log_{\frac{72}{19}}(1) = 0$. Therefore n^2 dominates the recursion term. So $T(n) = \Theta(n^2)$.

4.4

$T(n) = nT(\frac{n}{2}) + n^{\log_2 n}$. In this case, a=n, which means the Master method is not applicable (a depends on n).

4.5

$T(n) = 16T(\frac{n}{4}) + n^2$. In this case $a = 16, b = 4, f(n) = \Theta(n^2), \log_b(a) = \log_4(16) = 2$. Therefore n^2 matches $\log_b(a)$ so $T(n) = \Theta(n^2 \log n)$

4.6

$T(n) = 3T(\frac{n}{2}) + n^2$. In this case $a = 3, b = 2, f(n) = \Theta(n^2), \log_b(a) = \log_2(3) \approx 1.585 < 2$. Therefore $f(n)$ dominates so $T(n) = \Theta(n^2)$

4.7

$T(n) = T(\frac{n}{n-1}) + 1$. In this case b=n-1, so it depends on n, which means that the Master method is not applicable.

4.8

$T(n) = 4T(\frac{n}{16}) + \sqrt{n}$. In this case $a = 4, b = 16, f(n) = \Theta(\sqrt{n}), \log_b(a) = \log_{16}(4) = \frac{1}{2}$. $\sqrt{n} = n^{\frac{1}{2}}$, which matches $\log_{16}(4)$. So $T(n) = \Theta(\sqrt{n} \log n)$